

Table des matières

Labo 06 — De la commande à l’ordonnanceur	1
Avant de commencer — construire et démarrer la pile Airflow	2
Étape 1 — Régénérer les données jusqu’à mars 2026 : trois mois de plus	3
Étape 2 — Écrire la fenêtre de temps du pipeline	4
Étape 3 — Brancher la fenêtre et le partage des livraisons sur les étapes	6
Étape 4 — Charger une seule journée, et voir les garde-fous répondre	12
Étape 5 — Tester le rattrapage sans Airflow : deux tests de plus	13
Étape 6 — Allumer Airflow et lire les fichiers qui le composent	15
Étape 7 — Vérifier le pool de l’entrepôt, et voir l’erreur sans lui.....	18
Étape 8 — Écrire le DAG quotidien et lancer une journée	18
Étape 9 — Préparer un témoin, écrire le DAG mensuel et laisser rattraper	22
Étape 10 — Prouver que le rattrapage égale le chargement d’un bloc	25
Étape 11 — Versionner les DAG, la fenêtre et les tests.....	27
Récapitulatif	28
Dépannage	29
Annexe — Tout le code, d’un seul tenant.....	30

Labo 06 — De la commande à l’ordonnanceur

Labo · Module 6/12 — Formation Data Engineering

Ce qu’on fait. Trois mois de 2026 arrivent chez Boréal. Vous donnez d’abord au pipeline du Cours 05 la notion de tranche de temps, puis vous montez l’Airflow 3 de la plateforme, vous lisez les fichiers qui le composent, vous écrivez deux DAG, et vous les laissez rattraper les trois mois. Le labo se termine sur une preuve : l’état obtenu par trois exécutions successives est identique, ligne par ligne, à celui d’un chargement d’un seul bloc.

Il se fait en deux séances d’environ deux heures et demie, coupées après l’étape 5.

Trois terminaux dans ce labo, et chaque étape dit lequel. L’Anaconda Prompt sert à Python, à Git et au générateur. La ligne de commande DuckDB sert au SQL de vérification. Ubuntu, dans WSL 2, sert à Docker et à tout ce qui touche Airflow.

Comment lire les blocs de code de ce labo. Le Cours 05 faisait écrire des fichiers neufs, et un bloc valait pour un fichier entier. Celui-ci modifie des fichiers qui existent déjà, alors chaque bloc est précédé d'une ligne qui dit trois choses : quel fichier, quel geste, et à quel endroit. « Ajoutez sous... », « remplacez la fonction ... en entier par... », « le fichier en entier ». Quand la consigne dit « en entier », effacez ce que vous aviez et mettez ce qui suit à la place ; quand elle nomme une fonction, seule cette fonction change et le reste du fichier ne bouge pas.

- ☐ Sources régénérées jusqu'à 2026-03 : 1 989 fichiers, empreinte 20814937...
 - ☐ pipeline/fenetre.py écrit, et branché sur raw et sur la porte d'entrée
 - ☐ --livraisons en place : 14 livraisons pour le mensuel, 8 pour le quotidien
 - ☐ pytest rend 5 verts
 - ☐ L'image d'Airflow construite, la pile démarrée, les cinq services sains, et 01_de/dags créé
 - ☐ Le pool entrepot vérifié, avec une seule place
 - ☐ boreal_quotidien et boreal_mensuel inscrits, sans erreur d'importation
 - ☐ Le rattrapage des trois mois : 14 fichiers par exécution, 20 664 lignes en tout
 - ☐ La comparaison avec le témoin : 14 tables, 0 écart
 - ☐ Le tout versionné
-

Avant de commencer — construire et démarrer la pile Airflow

Ce cours allume Airflow, le seul service de la formation qui ne soit pas déjà là. Son image se construit une fois, à partir du `Dockerfile.airflow` reçu avec la plateforme au Cours 04 ; l'étape 6 dira ce que ce fichier contient et pourquoi. Le téléchargement pèse 3,18 Go et ne se refait jamais.

Airflow ne sert qu'à partir de l'étape 6 : si vous faites ce labo en deux séances, ce bloc peut attendre le début de la seconde. Dans l'Anaconda Prompt, depuis `Projet_Boreal` :

```
cd plateforme
wsl
```

Dans Ubuntu, depuis `plateforme` :

```
docker compose build airflow-init
docker compose up -d
docker compose ps
```

La construction télécharge l'image officielle d'Airflow et y ajoute DuckDB. Le démarrage prend moins d'une minute, le temps que la base de métadonnées d'Airflow se crée et que chaque service réponde à son contrôle de santé :

SERVICE	STATUS
airflow-apiserver	Up 22 seconds (healthy)
airflow-dag-processor	Up 21 seconds (healthy)
airflow-db	Up 46 seconds (healthy)
airflow-scheduler	Up 21 seconds (healthy)
airflow-triggerer	Up 21 seconds (healthy)
erp	Up 56 seconds

Si un service dit encore (health: starting), relancez `docker compose ps` une minute plus tard. Ce que la pile coûte, avec `docker stats --no-stream` : un peu plus de 1,1 Go de mémoire pour Airflow, là où sa documentation demande d'accorder 4 Go à Docker. `airflow-init`, lui, ne passe qu'une fois : il migre la base et crée le compte. Laissez la fenêtre Ubuntu ouverte.

Étape 1 — Régénérer les données jusqu'à mars 2026 : trois mois de plus

Théorie : Theorie_06.md, section 3.

Boréal a continué de tourner. Le générateur sait produire l'état de l'entreprise à une date plus tardive : c'est l'option `--jusqua`. Dans l'Anaconda Prompt, depuis `Data_Domain`, et non depuis `Projet_Boreal` :

```
cd ..
python _univers/boreal/generer_boreal.py --projet Projet_Boreal --jusqua 2026-03
```

Il réécrit `sources/` en entier, en 90 secondes environ. Les mois que vous aviez déjà sont identiques au bit près : le générateur est déterministe, et c'est ce qui rend ce labo rejouable. Le bilan final change :

erp/	51 extraits mensuels (et 2023-03_bis), 27 185 commandes, 82 842 postes, 29 782 véhicules à l'état du 2026-03-31
usine/	historien 182 jours, 550 087 lignes, 58 étiquettes ; 825 bons de travail ; 148 891 relevés du compteur
garanties/	51 extraits mensuels : 28 118 enregistrements, 57 583 visites, 7 343 lignes de réclamations
telemetrie/	974 jours, 204 485 messages reçus de 7 644 véhicules, dont 2 943 doublons
corpus/	34 documents publiés sur 34 au catalogue, 16 569 mots

1 989 fichiers, 177,5 Mo, empreinte globale 2081493759980946...

Les trois extraits mensuels de 2026 sont apparus, et l'historien est passé de 92 à 182 jours. Votre entrepôt, lui, n'a pas bougé : il en est resté à ce que le Cours 05 y a chargé. Retournez dans le projet :

```
cd Projet_Boreal
```

Piège — le générateur se lance depuis Data_Domain, jamais depuis Projet_Boreal. Lancé au mauvais endroit, il répond python: can't open file. Le dépannage du guide d'installation le dit aussi ; on le retrouve chaque fois qu'on régénère.

Étape 2 — Écrire la fenêtre de temps du pipeline

Théorie : Theorie_06.md, sections 3 et 5.

Le pipeline du Cours 05 charge « tout ce que le journal ignore ». Pour qu'une exécution d'Airflow puisse traiter sa tranche et rien d'autre, il lui faut une notion de fenêtre. Elle vaut d'être un module à elle : elle ne touche ni l'entrepôt ni les fichiers, elle répond seulement à une question, « ce lot tombe-t-il dans ma tranche ? ».

Dans l'Anaconda Prompt, depuis Projet_Boreal :

```
notepad 01_de\pipeline\fenetre.py
```

```
# -*- coding: utf-8 -*-  
"""fenetre.py : La tranche de temps qu'une exécution du pipeline traite.
```

```
  
Un Lot est ce que Le Cours 03 a écrit dans la colonne _Lot de La RAW : Le mois d'un extrait  
(2026-01), Le jour d'une archive de télémétrie (2026-01-14), ou « complet » pour un export livré  
en entier. La fenêtre dit quels fichiers regarder ; Le journal des chargements dit lesquels ont  
déjà été chargés. Les deux se complètent, et c'est le journal qui garde l'idempotence quand deux  
fenêtres se chevauchent.
```

```
  
Un export livré en entier entre dans toutes les fenêtres : il n'a pas de date, et son empreinte  
suffit à décider s'il a bougé. Un lot que la fenêtre ne sait pas lire y entre aussi : une tranche  
de temps ne doit jamais faire disparaître une livraison en silence.  
"""
```

```
from calendar import monthrange  
from dataclasses import dataclass  
from datetime import date
```

```
class DateIllisible(ValueError):  
    """Une borne de La ligne de commande n'est ni AAAA-MM ni AAAA-MM-JJ."""
```

```
def _bornes(texte: str) -> tuple:  
    """La première et la dernière journée que « 2026-01 » ou « 2026-01-14 » couvre."""  
    morceaux = texte.split("-")
```

```

try:
    if len(morceaux) == 2:
        annee, mois = int(morceaux[0]), int(morceaux[1])
        return date(annee, mois, 1), date(annee, mois, monthrange(annee, mois)[1])
    if len(morceaux) == 3:
        jour = date(int(morceaux[0]), int(morceaux[1]), int(morceaux[2]))
        return jour, jour
except ValueError as erreur:
    raise DateIllisible(f"« {texte} » n'est pas une date : {erreur}") from erreur
raise DateIllisible(f"« {texte} » n'est ni AAAA-MM ni AAAA-MM-JJ.")

```

```
@dataclass(frozen=True)
```

```
class Fenetre:
```

```
    """La tranche traitée par une exécution. Sans borne, elle contient tout."""
```

```
    depuis: date = None
```

```
    jusquà: date = None
```

```
@classmethod
```

```
def depuis_la_ligne_de_commande(cls, depuis: str = None, jusquà: str = None) -> "Fenetre":
```

```
    """« --depuis 2026-01 --jusquà 2026-01 » donne le mois de janvier en entier."""
```

```
    return cls(depuis=_bornes(depuis)[0] if depuis else None,
               jusquà=_bornes(jusquà)[1] if jusquà else None)
```

```
@property
```

```
def ouverte(self) -> bool:
```

```
    """Vrai quand aucune borne n'a été donnée : l'étape travaille comme au Cours 05."""
```

```
    return self.depuis is None and self.jusquà is None
```

```
def contient(self, lot: str) -> bool:
```

```
    """Le lot d'un fichier tombe-t-il dans la tranche ?
```

```
    Un lot relivré garde sa date : 2023-03_bis reste le mois de mars 2023.
```

```
    """
```

```
    if self.ouverte:
```

```
        return True
```

```
    try:
```

```
        debut, fin = _bornes(lot.split("_")[0])
```

```
    except DateIllisible:
```

```
        return True
```

```
    return not (self.jusquà is not None and debut > self.jusquà)
```

```

        or self.depuis is not None and fin < self.depuis)

def __str__(self) -> str:
    if self.ouverte:
        return "tout l'historique"
    return f"du {self.depuis or 'début'} au {self.jusqua or 'dernier jour livré'}"

```

Trois décisions sont inscrites dans ce fichier, et chacune se paierait cher dans l'autre sens.

La décision	Pourquoi
une borne de mois vaut tout le mois	--jusqua 2026-01 ne doit pas s'arrêter au premier janvier
« complet » et l'inconnu passent	une tranche de temps ne fait pas disparaître une livraison en silence
une borne illisible lève	mieux vaut un arrêt net qu'un filtre qui écarte tout sans le dire

Étape 3 — Brancher la fenêtre et le partage des livraisons sur les étapes

Théorie : Theorie_06.md, sections 5 et 7.

Deux choses manquent au pipeline pour qu'un DAG puisse lui parler. La fenêtre, qui vient d'être écrite, et le moyen de nommer les livraisons qu'une exécution possède. La seconde se comprend mieux avec un exemple : une fenêtre du mois de mars attrape aussi les archives quotidiennes de mars. Si le DAG mensuel et le DAG quotidien ne se partagent pas les livraisons, ils se disputent le même travail.

Dans 01_de\pipeline\raw.py, ajoutez cette ligne en tête du fichier, sous l'importation de entrepot :

```
from .fenetre import Fenetre
```

Toujours dans raw.py, remplacez la fonction a_charger en entier par le bloc qui suit. Il contient aussi le début de charger_une, qui existe déjà : de celle-là, seules la ligne def, qui gagne un argument, et la première ligne du corps, qui le transmet, changent. Le reste du corps de charger_une ne bouge pas.

```

def a_charger(con, table: str, motif: str, regle: str, fenetre: Fenetre = None) -> list:
    """Les fichiers du motif que le journal ne connaît pas, ou connaît sous une autre empreinte.

    Deux filtres, dans cet ordre, et l'ordre compte. La fenêtre écarte d'abord les fichiers dont le
    lot tombe hors de la tranche : elle ne lit rien, elle regarde des noms. Le journal écarte
    ensuite ceux qu'il a déjà chargés sous la même empreinte : lui doit lire chaque fichier restant
    pour en calculer l'empreinte. Filtrer par la fenêtre d'abord, c'est ne pas relire 1 572 fichiers
    pour en charger 14.
    """
    fenetre = fenetre or Fenetre()

```

```

connus = deja_charge(con, table)
fichiers = [p.as_posix() for p in Path(".").glob(motif)]
dans_la_tranche = sorted(f for f in fichiers if fenetre.contient(lot_du(f, regle)))
return [(f, sceau, lot_du(f, regle)) for f in dans_la_tranche
        if connus.get(f) != (sceau := empreinte(f))]

```

```

def charger_une(con, table: str, motif: str, forme: str, regle: str,
               fenetre: Fenetre = None) -> tuple:
    """Charge les fichiers nouveaux ou modifiés d'une livraison. Rend (fichiers, Lignes)."""
    nouveaux = a_charger(con, table, motif, regle, fenetre)

```

Le bloc qui suit mêle trois gestes, et il vaut de les distinguer avant de taper. La classe LivraisonInconnue et la fonction sources_de sont neuves, elles n'existent nulle part dans le paquet : vous les ajoutez juste au-dessus de executer. executer, lui, existe déjà : vous en remplacez l'en-tête et la ligne for, et rien d'autre.

```

class LivraisonInconnue(ValueError):
    """Un préfixe de livraison qui n'existe pas, passé par --livraisons."""

def sources_de(livraisons=None) -> list:
    """Les livraisons demandées, par leur préfixe : « erp », « telemetrie »... Toutes par défaut.

    Chaque DAG nomme ainsi ce qu'il possède : Le mensuel prend l'ERP et Les garanties, le quotidien
    prend l'usine, la télémétrie, la météo et le corpus. Deux DAG qui se partagent les livraisons ne
    se marchent pas dessus, et chacun se lit sans deviner ce que l'autre fait.

    """
    if not livraisons:
        return LIVRAISONS
    connus = sorted({table.split("_")[0] for table, _, _ in LIVRAISONS})
    inconnus = [nom for nom in livraisons if nom not in connus]
    if inconnus:
        # Une erreur du domaine, jamais un SystemExit : celui-ci n'est pas une Exception, il
        # traverserait le gestionnaire d'étape sans fermer sa ligne ni passer par le journal.
        raise LivraisonInconnue(f"Livraison inconnue : {' '.join(inconnus)}. "
                                f"Connues : {' '.join(knowns)}.")
    return [ligne for ligne in LIVRAISONS if ligne[0].split("_")[0] in livraisons]

def executer(con, fenetre: Fenetre = None, livraisons=None) -> int:
    """Charge les livraisons demandées. Rend le nombre de lignes chargées à ce passage."""
    fenetre = fenetre or Fenetre()

```

```
total_fichiers = total_lignes = 0
for table, motif, forme, regle in sources_de(livraisons):
    fichiers, lignes = charger_une(con, table, motif, forme, regle, fenetre)
```

Le reste du corps de executer ne bouge pas : la boucle garde ses deux LOG.info, ses compteurs et son return.

Les quatre autres étapes reçoivent les deux arguments et n'en font rien : la porte d'entrée les passe à toutes, et chacune prend ce qui la concerne. Dans bronze.py, corpus.py, erp.py et meteo.py, remplacez la ligne def executer et la docstring qui la suit par ce qui est écrit ci-dessous, une par fichier. Le reste de chaque fonction ne bouge pas.

Une seule des quatre reçoit la connexion sans valeur par défaut, bronze.py, parce qu'elle s'en sert. Les trois autres gardent le con=None qu'elles avaient déjà : elles ne touchent pas à l'entrepôt, et recopier la signature de bronze.py chez elles casserait leur appel.

bronze.py

```
def executer(con, fenetre=None, livraisons=None) -> int:
    """Refait le bronze de l'historien et de la télémétrie. Rend le total des lignes écrites.
```

La fenêtre ne s'applique pas ici : Le COPY du Cours 02 réécrit ses partitions avec OVERWRITE, et son contrôle compare les totaux des deux côtés. Reconstruire tout coûte quelques secondes et garde la garantie ; ne réécrire qu'un mois demanderait un contrôle par partition.

corpus.py

```
def executer(con=None, fenetre=None, livraisons=None) -> int:
    """Dépose le corpus dans le bronze. Rend le nombre de fichiers déposés à ce passage.
```

La fenêtre ne s'applique pas ici : un document n'a pas de lot, et c'est son empreinte qui dit s'il a bougé.

erp.py

```
def executer(con=None, fenetre=None, livraisons=None) -> int:
    """Extrait les neuf tables de la base. Rend le total des lignes lues.
```

La fenêtre ne s'applique pas ici : L'ERP livre une base, pas des fichiers datés, et c'est justement ce que le Cours 04 oppose aux extraits mensuels.

meteo.py

```
def executer(con=None, fenetre=None, livraisons=None) -> int:
    """Tire les observations de la station configurée. Rend le nombre de jours reçus.
```

La fenêtre ne s'applique pas ici : L'API porte ses propres dates, dans config.toml. Les brancher

sur la fenêtre est la question pratique 6 de ce cours.
"""

Quatre phrases qui disent la même chose de quatre façons, et c'est voulu : celui qui ouvrira `erp.py` dans six mois n'ira pas chercher l'explication dans `bronze.py`.

Reste la porte d'entrée. Six endroits y changent, dispersés dans le fichier : l'importation, le tableau des étapes, les trois arguments de la ligne de commande, la façon dont résoudre et `--etapes` lisent le tableau, la construction de la fenêtre, et l'appel de chaque étape. Plutôt que de les décrire un par un, voici `01_de\pipeline\__main__.py` en entier. Effacez ce que vous aviez et mettez ceci à la place :

```
# -*- coding: utf-8 -*-  
"""__main__.py : La porte d'entrée du pipeline de Boréal (Cours 05).
```

Ce qu'il fait. Il lit la ligne de commande, ouvre le journal et l'entrepôt une seule fois, puis lance les étapes demandées, chacune dans son encadrement : une ligne dans `pipeline.executions` au début, sa durée et son compte à la fin, et l'erreur inscrite si elle échoue.

```
python -m pipeline bronze raw corpus      Les étapes nommées, dans cet ordre  
python -m pipeline tout                   Les étapes du chemin hors ligne, dans le bon ordre  
python -m pipeline raw --bavard           Le journal descend au niveau DEBUG  
python -m pipeline --etapes               La liste des étapes, sans rien lancer
```

```
python -m pipeline raw --depuis 2026-01 --jusqua 2026-01      un seul mois (Cours 06)  
python -m pipeline raw --depuis 2026-01-14 --jusqua 2026-01-14 une seule journée  
python -m pipeline raw --livraisons erp,garanties             seulement ces livraisons
```

La règle qu'il illustre. Un pipeline a une seule porte d'entrée. Le DAG du Cours 06 appellera cette commande, tâche par tâche ; le kit de départ d'Analytics Engineering la rejouera sans Airflow ; le conteneur la lance comme point d'entrée de l'image. Trois façons d'appeler, un seul chemin de code.

Le code de sortie compte autant que le journal : 0 quand tout est passé, 1 quand une étape a échoué. C'est ce que l'ordonnanceur lit, et c'est pour lui que `main()` ne laisse jamais filer une exception.

Deux étapes ne sont pas dans « tout » : `erp`, qui demande le conteneur de la base et le pilote `psqlODBC`, et `meteo`, qui demande le réseau. Un pipeline rejouable hors ligne ne les suppose pas.
"""

```
import argparse  
import os  
import sys
```

```
from . import bronze, corpus, erp, meteo, raw  
from .config import projet
```

```

from .entrepot import ouvrir
from .fenetre import DateIllisible, Fenetre
from .journal import LOG, demarrer, etape

# Le nom de l'étape, ce qu'elle fait, si « tout » la prend, et si une fenêtre la restreint.
# La dernière colonne est celle du Cours 06 : seule l'étape raw travaille livraison par livraison,
# donc elle seule peut se limiter à une tranche de temps. Les autres reconstruisent tout, par
# conception : le bronze réécrit ses partitions (OVERWRITE, Cours 02), le corpus compare des
# empreintes, l'ERP lit une base entière et la météo appelle une API qui porte ses propres dates.
ETAPES = {
    "bronze": (bronze.executer, "les CSV et le JSON de l'usine, rangés en Parquet", True, False),
    "raw": (raw.executer, "les 22 livraisons chargées dans la couche RAW", True, True),
    "corpus": (corpus.executer, "la documentation déposée telle quelle dans le bronze", True, False),
    "erp": (erp.executer, "les neuf tables de la base de l'ERP, par ODBC", False, False),
    "meteo": (meteo.executer, "les observations d'ECCC, par son API", False, False),
}

def lire_arguments(arguments=None) -> argparse.Namespace:
    analyseur = argparse.ArgumentParser(
        prog="python -m pipeline",
        description="Le pipeline de Véhicules Boréal inc. : une étape, plusieurs, ou tout.")
    analyseur.add_argument("--etapes", nargs="*", metavar="ÉTAPE",
        help="les étapes à lancer, ou « tout »")
    analyseur.add_argument("--bavard", action="store_true", help="journal au niveau DEBUG")
    analyseur.add_argument("--etapes", dest="lister", action="store_true",
        help="liste les étapes et ce qu'elles font, sans rien lancer")
    analyseur.add_argument("--depuis", metavar="AAAA-MM[-JJ]",
        help="premier lot traité ; sans lui, tout l'historique")
    analyseur.add_argument("--jusqua", metavar="AAAA-MM[-JJ]",
        help="dernier lot traité ; sans lui, jusqu'au dernier livré")
    analyseur.add_argument("--livraisons", metavar="erp,garanties",
        help="les livraisons traitées, par préfixe ; sans elles, les 22")
    return analyseur.parse_args(arguments)

def resoudre(demandees: list) -> list:
    """La liste des étapes à lancer, dans l'ordre. « tout » prend celles du chemin hors ligne."""
    if demandees == ["tout"]:
        return [nom for nom, (_, _, dans_tout, _) in ETAPES.items() if dans_tout]
    inconnues = [nom for nom in demandees if nom not in ETAPES]
    if inconnues:

```

```

        raise SystemExit(f"Étape inconnue : {'', '.join(inconnues)}. "
                        f"Connues : {'', '.join(ETAPES)}, ou « tout ».")
    return demandees

def main(arguments=None) -> int:
    options = lire_arguments(arguments)
    if options.lister or not options.etapes:
        print("Étapes du pipeline :")
        for nom, (_, quoi, dans_tout, fenetree) in ETAPES.items():
            marques = ([] if dans_tout else ["hors de « tout »"]) + \
                (["fenêtre"] if fenetree else ["reconstruit tout"])
            print(f" {nom:8} {quoi} ({', '.join(marques)})")
        return 0

    try:
        fenetre = Fenetre.depuis_la_ligne_de_commande(options.depuis, options.jusqua)
    except DateIllisible as erreur:
        raise SystemExit(str(erreur))
    if fenetre.depuis and fenetre.jusqua and fenetre.depuis > fenetre.jusqua:
        raise SystemExit(f"Fenêtre vide : --depuis {options.depuis} vient après "
                        f"--jusqua {options.jusqua}.")

    livraisons = [nom.strip() for nom in options.livraisons.split(",")] if options.livraisons else None

    a_lancer = resoudre(options.etapes)
    # Le pipeline travaille depuis Projet_Boreal, d'où qu'on l'appelle : Les chemins des scripts SQL
    # et le lignage de la RAW s'écrivent alors toujours de la même façon.
    os.chdir(projet())
    if options.bavard:
        os.environ["PIPELINE_JOURNAL_NIVEAU"] = "DEBUG"
    demarrer()
    LOG.info("pipeline : %s, depuis %s", "", ".join(a_lancer), projet())
    LOG.info("fenêtre : %s", fenetre)
    LOG.info("livraisons : %s", "", ".join(livraisons) if livraisons else "toutes")
    if not fenetre.ouverte:
        ignorees = [nom for nom in a_lancer if not ETAPES[nom][3]]
        if ignorees:
            # Un avertissement, pas une erreur : un DAG quotidien lance bronze et raw dans la même
            # commande, et il est normal que seule la seconde se restreigne.
            LOG.warning("fenêtre ignorée par %s : cette étape reconstruit tout",
                        "", ".join(ignorees))

```

```

con = ouvrir()
try:
    for nom in a_lancer:
        action = ETAPES[nom][0]
        with etape(con, nom) as trace:
            # Une étape rend un entier, ce qu'elle a écrit. Le convertir ici fait dire à une
            # étape mal écrite ce qu'elle a fait de travers, plutôt qu'à DuckDB deux lignes
            # plus loin, dans une erreur de conversion de type.
            resultat = action(con, fenetre, livraisons)
            trace.lignes = None if resultat is None else int(resultat)
except Exception as erreur:
    LOG.error("pipeline arrêté : %s", erreur)
    return 1
finally:
    con.close()
LOG.info("pipeline : terminé")
return 0

if __name__ == "__main__":
    sys.stdout.reconfigure(encoding="utf-8", errors="replace")
    sys.exit(main())

```

Vérifiez d’abord que la porte d’entrée se lit :

```

cd 01_de
python -m pipeline --etapes

```

Étapes du pipeline :

```

bronze  les CSV et le JSON de l'usine, rangés en Parquet   (reconstruit tout)
raw     les 22 livraisons chargées dans la couche RAW     (fenêtre)
corpus  la documentation déposée telle quelle dans le bronze (reconstruit tout)
erp     les neuf tables de la base de l'ERP, par ODBC      (hors de « tout », reconstruit tout)
meteo   les observations d'ECCC, par son API               (hors de « tout », reconstruit tout)

```

Étape 4 — Charger une seule journée, et voir les garde-fous répondre

Théorie : Theorie_06.md, section 5.

Le premier essai porte sur une seule journée de télémétrie, celle du 14 janvier 2026. Toujours dans l’Anaconda Prompt, depuis 01_de :

```

python -m pipeline raw --livraisons telemetrie --depuis 2026-01-14 --jusqua 2026-01-14

```

```
pipeline : raw, depuis C:\...\Data_Domain\Projet_Boreal
fenêtre : du 2026-01-14 au 2026-01-14
livraisons : telemetrie
    telemetrie_sorties          1 fichier(s),      222 lignes
    RAW : 1 fichier(s) chargé(s) à ce passage
étape raw (dc01073f) : fin, 222 lignes en 0.1 s
```

Un fichier, 222 messages, un dixième de seconde. Sur 1 989 fichiers dans sources/, la fenêtre en a retenu un, et le journal a confirmé qu'il était nouveau. Relancez la même commande :

```
    telemetrie_sorties          à jour
    RAW : 0 fichier(s) chargé(s) à ce passage
étape raw (aa65bc92) : fin, 0 lignes en 0.0 s
```

Rien. C'est le journal des chargements qui parle ici, pas la fenêtre : le fichier est dans la tranche, mais son empreinte est déjà connue.

Trois commandes fautives, pour voir les garde-fous répondre :

```
python -m pipeline raw --livraisons erp,garantie
Livraison inconnue : garantie. Connues : corpus, erp, garanties, meteo, telemetrie, usine.

python -m pipeline raw --depuis 2026-13
« 2026-13 » n'est pas une date : month must be in 1..12

python -m pipeline raw --depuis 2026-03 --jusqua 2026-01
Fenêtre vide : --depuis 2026-03 vient après --jusqua 2026-01.
```

À retenir — un garde-fou qui parle vaut mieux qu'un filtre muet. Sans ces trois arrêts, un « s » oublié ou deux bornes inversées donneraient une exécution verte qui n'aurait rien chargé. C'est le pire résultat possible : faux et silencieux.

Étape 5 — Tester le rattrapage sans Airflow : deux tests de plus

Théorie : Theorie_06.md, sections 4 et 5.

Le Cours 05 a laissé trois tests. Deux de plus vérifient ce que le cours vient d'ajouter. Ouvrez 01_de\tests\test_pipeline.py, ajoutez l'importation :

```
from pipeline.fenetre import DateIllisible, Fenetre
```

puis ces deux tests à la fin du fichier :

```
def test_la_fenetre_ne_retient_que_les_lots_de_sa_tranche():
    """Un mois, un jour, une relivraison, un export complet : chacun à sa place, sans base ni fichier."""
```

```

janvier = Fenetre.depuis_la_ligne_de_commande("2026-01", "2026-01")
assert janvier.contient("2026-01")
assert not janvier.contient("2025-12")
assert not janvier.contient("2026-02")

# Un lot du jour : La fenêtre du mois le prend, celle d'une autre journée non.
assert janvier.contient("2026-01-14")
assert not Fenetre.depuis_la_ligne_de_commande("2026-01-14", "2026-01-14").contient("2026-01-15")

# Une relivraison garde la date de son mois : 2023-03_bis est bien du mois de mars 2023.
mars = Fenetre.depuis_la_ligne_de_commande("2023-03", "2023-03")
assert mars.contient("2023-03_bis")

# Ce qui n'a pas de date entre partout : c'est le journal qui décide ensuite.
assert janvier.contient("complet")
assert janvier.contient("un_lot_inattendu")

# Sans borne, la fenêtre est ouverte : Le pipeline se comporte comme au Cours 05.
assert Fenetre().ouverte and Fenetre().contient("2019-07")

# Une borne illisible se dit tout de suite, plutôt que de filtrer de travers.
with pytest.raises(DateIllisible):
    Fenetre.depuis_la_ligne_de_commande("janvier 2026")

def test_trois_fenetres_chargent_ce_qu'une_seule_aurait_charge(projet):
    """Le rattrapage mois par mois donne la même RAW qu'un chargement d'un bloc. C'est la promesse
    que le DAG mensuel tient, et elle vaut d'être vérifiée sans Airflow."""
    for mois, lignes in (("2026-01", ["0000000001,100", "0000000002,200"]),
                        ("2026-02", ["0000000003,300"]),
                        ("2026-03", ["0000000004,400", "0000000005,500"])):
        ecrire_extrait(projet, mois, lignes)
    motif, table = "sources/erp/extraits/*/VBAK.csv", "erp_vbak"

    con = ouvrir()
    try:
        # Le rattrapage : une exécution par mois, chacune dans sa tranche.
        for mois in ("2026-01", "2026-02", "2026-03"):
            fenetre = Fenetre.depuis_la_ligne_de_commande(mois, mois)
            fichiers, _ = raw.charger_une(con, table, motif, "csv", "dossier", fenetre)
            assert fichiers == 1, f"{mois} aurait dû charger son seul fichier"

```

```

par_lot = dict(con.execute(
    f"SELECT _lot, COUNT(*) FROM raw.{table} GROUP BY _lot").fetchall())
assert par_lot == {"2026-01": 2, "2026-02": 1, "2026-03": 2}

# Une exécution de plus, fenêtre ouverte : Le journal a tout vu, il ne reste rien.
assert raw.charger_une(con, table, motif, "csv", "dossier") == (0, 0)

# Et deux fenêtres qui se chevauchent ne doublent rien : c'est le journal qui garde
# l'idempotence, pas la fenêtre.
large = Fenetre.depuis_la_ligne_de_commande("2026-01", "2026-03")
assert raw.charger_une(con, table, motif, "csv", "dossier", large) == (0, 0)
assert con.execute(f"SELECT COUNT(*) FROM raw.{table}").fetchone()[0] == 5
finally:
    con.close()

```

Le fichier s'arrête là. La commande qui suit se tape dans l'Anaconda Prompt, depuis 01_de, et non dans le fichier :

```

pytest
..... [100%]
5 passed in 0.49s

```

Cinq tests, une demi-seconde, sans les 177 Mo de Boréal et sans conteneur. Le second test est le plus important du cours : il vérifie la promesse du rattrapage avant qu'Airflow n'entre en scène, si bien que le jour où le DAG se comportera mal, vous saurez que le pipeline, lui, tient.

Fin de la première séance. Le pipeline sait maintenant traiter une tranche de temps, et il le prouve sans Airflow : c'est la moitié du cours, et elle se tient debout seule. La suite allume l'ordonnanceur et demande une seconde séance d'environ deux heures et demie. Arrêtez ici si vous n'avez pas ce temps devant vous.

Étape 6 — Allumer Airflow et lire les fichiers qui le composent

Théorie : Theorie_06.md, section 1.

La pile est montée depuis le bloc d'ouverture de ce labo : l'image est construite, la base de métadonnées est créée, le compte existe. Si vous l'avez arrêtée entre les deux séances, rallumez-la. Depuis Projet_Boreal, dans l'Anaconda Prompt :

```

cd plateforme
wsl

```

Dans Ubuntu, depuis plateforme :

```

docker compose start
docker compose ps

```

Le redémarrage prend une vingtaine de secondes, le temps que chaque service réponde à son contrôle de santé :

SERVICE	STATUS
airflow-apiserver	Up 23 seconds (healthy)
airflow-dag-processor	Up 23 seconds (healthy)
airflow-db	Up 42 seconds (healthy)
airflow-scheduler	Up 23 seconds (healthy)
airflow-triggerer	Up 23 seconds (healthy)
erp	Up 42 seconds

Piège — Ubuntu qui s’arrête dans votre dos. Comme au Cours 04, gardez cette fenêtre Ubuntu ouverte jusqu’à la fin du labo. `restart: unless-stopped` fait repartir les conteneurs quand Ubuntu redémarre, mais le temps que tout redevienne sain, une exécution en cours est perdue.

Ouvrez `http://localhost:8080` dans un navigateur, sous Windows : l’interface répond, le compte est `airflow / airflow`, et la liste des DAG est vide. Elle le restera tant que le dossier où Airflow les cherche n’existe pas. Ce dossier est à vous, et c’est le premier geste de ce cours qui touche votre dépôt. Dans une autre fenêtre de l’Anaconda Prompt, depuis `Projet_Boreal` :

```
mkdir 01_de\dags
```

Airflow le trouvera seul, sans redémarrage : son lecteur de DAG relit le dossier en continu.

Reste à comprendre ce qui tourne. Les fichiers de la plateforme sont arrivés tout prêts, comme en entreprise, et vous ne les modifiez pas ; mais un ingénieur de données doit savoir les lire. Ouvrez-les en lecture, depuis `Projet_Boreal` :

```
notepad plateforme\Dockerfile.airflow
```

```
FROM apache/airflow:3.3.1
```

```
# pip s'exécute sous l'utilisateur airflow, jamais sous root : L'image officielle le refuse.
```

```
USER airflow
```

```
RUN pip install --no-cache-dir duckdb==1.5.5
```

Trois lignes, et une leçon. Airflow a besoin de DuckDB, puisque ses tâches appelleront le pipeline. La documentation officielle propose d’installer les dépendances au démarrage du conteneur, par une variable d’environnement, et écrit elle-même que c’est réservé aux essais : l’installation se refait à chaque démarrage, et les journaux du cours s’ouvriraient sur un avertissement en rouge. Un `Dockerfile` règle la question une fois pour toutes. La version de DuckDB y est la même que dans le `Dockerfile` du pipeline du Cours 05, et ce n’est pas un détail : les deux écrivent le même fichier d’entrepôt.

Puis le `docker-compose.yml`, qui décrit la pile :

```
notepad plateforme\docker-compose.yml
```

Cinq services d’Airflow, plus la base de l’ERP du Cours 04. Le premier bloc est la définition que les services partagent, écrivez une fois sous l’ancrage `&airflow-commun` et recopiez ensuite par `<<: *airflow-commun` :

Ce que la définition partagée porte	Ce que ça veut dire
AIRFLOW__CORE__EXECUTOR: LocalExecutor	les tâches s'exécutent dans des processus de l'ordonnanceur, sur cette machine. Ni Redis, ni ouvrier Celery : rien ici ne s'exécute sur plusieurs machines
AIRFLOW__DATABASE__SQL_ALCHEMY_CONN	la base de métadonnées d'Airflow, un PostgreSQL à lui, distinct de celui de l'ERP
AIRFLOW__CORE__DAGS_FOLDER: /projet/01_de/dags	Airflow lit vos DAG dans votre dépôt, à travers le montage du projet. Aucun montage propre aux DAG : Docker créerait sinon le dossier avant vous
AIRFLOW__CORE__LOAD_EXAMPLES: 'false'	l'interface reste vide des dizaines d'exemples livrés avec Airflow
PIPELINE_PROJET: /projet et PYTHONPATH: /projet/01_de	le paquet pipeline du Cours 05 est importable dans le conteneur, et il y trouve le projet
../projet	tout Projet_Boreal est monté : les sources, le lac, l'entrepôt et votre dépôt
./airflow/logs et ./airflow/config	les journaux et la configuration d'Airflow, du côté de la plateforme et non du vôtre

Les cinq services, ensuite, chacun une commande d'Airflow avec son contrôle de santé :

Service	Ce qu'il fait
airflow-db	la base de métadonnées : l'état de chaque exécution et de chaque tâche
airflow-init	passé une seule fois, au premier démarrage : migre la base et crée le compte
airflow-apiserver	l'interface web et l'API, sur le port 8080
airflow-scheduler	décide quoi lancer et quand, puis exécute les tâches
airflow-dag-processor	relit les fichiers de DAG et les inscrit dans la base
airflow-triggerer	tient les tâches en attente sans bloquer une place d'exécution

Pourquoi — cinq conteneurs, et pas sept. Le compose officiel d'Airflow ajoute Redis et un ouvrier Celery, qui servent à répartir les tâches sur plusieurs machines. Rien dans cette formation ne s'exécute ailleurs que sur votre poste. La pile mesurée ici demande un peu plus de 1,1 Go, là où la documentation d'Airflow demande d'accorder 4 Go à Docker pour sa pile complète.

Étape 7 — Vérifier le pool de l'entrepôt, et voir l'erreur sans lui

Théorie : Theorie_06.md, section 7.

Vous allez avoir deux DAG qui écrivent le même fichier DuckDB, et rien dans un graphe ne voit ce qui se passe dans l'autre. La plateforme a posé le verrou d'office, au premier démarrage de la pile. Vérifiez-le, dans Ubuntu depuis plateforme :

```
docker compose exec airflow-scheduler airflow pools list
```

pool	slots	description	include_deferred	team_name
default_pool	128	Default pool	False	None
entrepot	1	Un seul écrivain sur boreal.duckdb	False	None

Une seule place. Toute tâche qui déclarera ce pool attendra son tour, quel que soit le DAG où elle vit. Voici ce que donne l'oubli, relevé sur le poste de l'auteur en laissant les deux DAG démarrer en même temps sans le pool :

```
IO Error: Could not set lock on file "/projet/plateforme/entrepot/boreal.duckdb":  
Conflicting lock is held in /usr/python/bin/python3.13 (PID 94).
```

La tâche meurt, la nouvelle tentative repart cinq minutes plus tard, et le DAG affiche un rouge dont la cause n'est pas dans son propre graphe. C'est la limite de D67, un seul écrivain à la fois sur l'entrepôt, que l'ordonnanceur doit respecter à la place du moteur.

À retenir — un pool est une ressource, pas une dépendance. Une dépendance ordonne deux tâches d'un même graphe. Un pool borne le nombre de tâches qui touchent une même ressource, à travers tous les DAG de l'instance. C'est le seul outil d'Airflow qui voit plus loin qu'un graphe.

Étape 8 — Écrire le DAG quotidien et lancer une journée

Théorie : Theorie_06.md, sections 2, 3, 6 et 8.

Le premier DAG reprend le chemin froid du Cours 05 et le découpe en tâches. Il commence par la fonction qu'appellera l'alerte de retard. Dans l'Anaconda Prompt, depuis Projet_Boreal :

```
notepad 01_de\dags\boreal_alertes.py
```

```
# -*- coding: utf-8 -*-  
"""boreal_alertes.py : ce qu'on fait quand une exécution dépasse son délai.
```

Une alerte de retard n'est pas une reprise. Personne ne relance rien : on signale, parce qu'un chargement qui met trois heures au lieu de dix minutes finit peut-être par passer, et c'est justement ce silence qui coûte cher. En production, cette fonction enverrait un courriel ou un message ; dans le labo, elle écrit, et c'est la même mécanique.

Une fonction ordinaire s'enveloppe dans SyncCallback. AsyncCallback, celui que la documentation montre en exemple, attend une coroutine : lui donner cette fonction fait échouer la lecture du DAG sur « Provided callback ... is not awaitable », et le DAG n'apparaît pas du tout dans l'interface.

```
import logging
```

```
journal = logging.getLogger(__name__)
```

```
def prevenir_retard(**contexte) -> None:
```

```
    """Écrit le dépassement dans le journal. Les valeurs arrivent déjà remplies par Airflow."""
```

```
    journal.warning("RETARD : %s", contexte.get("message", contexte))
```

Puis le DAG :

```
notepad 01_de\dags\boreal_quotidien.py
```

```
# -*- coding: utf-8 -*-
```

```
"""boreal_quotidien.py : Le chemin froid de Boréal, une fois par jour.
```

Quatre tâches, dans l'ordre : Le bronze de l'usine, la couche RAW limitée à la journée de l'exécution, le dépôt du corpus, puis la place que dbt prendra. Chacune appelle la porte d'entrée du pipeline. Airflow ordonnance, le pipeline travaille.

Un DAG mélange deux sortes de tâches, et il faut savoir laquelle on écrit. « raw » est incrémentale : elle reçoit la date de son exécution et ne charge que les livraisons de cette journée.

« bronze » reconstruit tout à chaque passage, parce que le COPY du Cours 02 réécrit ses partitions et vérifie ses totaux. Les deux sont rejouables, par des chemins différents.

```
"""
```

```
from __future__ import annotations
```

```
from datetime import timedelta
```

```
import pendulum
```

```
from airflow.sdk import DeadlineAlert, DeadlineReference, SyncCallback, dag
```

```
from airflow.providers.standard.operators.bash import BashOperator
```

```
from airflow.providers.standard.operators.empty import EmptyOperator
```

```
from airflow.timetables.interval import CronDataIntervalTimetable
```

```
from boreal_alertes import prevenir_retard
```

```
# Le pipeline se lance depuis le projet monté ; PIPELINE_PROJET et PYTHONPATH viennent du compose.
```

```
PIPELINE = "cd /projet/01_de && python -m pipeline"
```

```

# Le pool à une place qui garde l'entrepôt. Toute tâche qui appelle le pipeline le prend, dans ce
# DAG comme dans le mensuel : DuckDB n'accepte qu'un écrivain à la fois sur son fichier, et le
# nombre d'exécutions simultanées ne protège un DAG que de lui-même.
ENTREPOT = "entrepot"

```

```

@dag(
    dag_id="boreal_quotidien",
    # L'intervalle de données, écrit en toutes lettres : depuis Airflow 3, « @daily » tout court ne
    # découpe plus de tranche, et l'exécution de minuit chargerait la journée qui commence.
    schedule=CronDataIntervalTimetable("@daily", timezone="UTC"),
    start_date=pendulum.datetime(2026, 3, 1, tz="UTC"),
    # Sans rattrapage : l'historique a été chargé d'un bloc au cours 05, et personne ne veut
    # 1 500 exécutions quotidiennes à la première mise en route. Le DAG mensuel, lui, rattrape.
    catchup=False,
    max_active_runs=1,
    # Deux essais, cinq minutes d'attente : une écriture DuckDB refusée parce qu'un autre processus
    # tenait l'entrepôt passe au second essai, une erreur de SQL non.
    default_args={"retries": 1, "retry_delay": timedelta(minutes=5)},
    deadline=DeadlineAlert(
        reference=DeadlineReference.DAGRUN_QUEUED_AT,
        interval=timedelta(minutes=30),
        callback=SyncCallback(
            prevenir_retard,
            kwargs={"message": "boreal_quotidien du {{ ds }} n'a pas fini en 30 minutes"},
        ),
    ),
    tags=["boreal", "chemin froid"],
    doc_md=__doc__,
)
def boreal_quotidien():
    bronze = BashOperator(
        task_id="bronze",
        bash_command=f"{PIPELINE} bronze",
        pool=ENTREPOT,
    )

    # La seule tâche fenêtrée du DAG. {{ ds }} est la date logique de l'exécution, le début de sa
    # tranche, et non le jour où elle tourne : rejouer le 15 mars en juin recharge le 15 mars. C'est
    # aussi la bonne date pour une exécution lancée à la main. Les livraisons nommées sont celles
    # que ce DAG possède ; l'ERP et les garanties arrivent par mois.
    raw = BashOperator(

```

```

    task_id="raw",
    bash_command=f"{PIPELINE} raw --livraisons usine,telemetrie,meteo,corpus "
                f"--depuis {{{ ds }}} --jusqua {{{ ds }}}",
    pool=ENTREPOT,
)

corpus = BashOperator(
    task_id="corpus",
    bash_command=f"{PIPELINE} corpus",
    pool=ENTREPOT,
)

# La place de dbt, que La formation Analytics Engineering remplira par « dbt build ».
dbt_build = EmptyOperator(task_id="dbt_build")

bronze >> raw >> corpus >> dbt_build

```

boreal_quotidien()

Un fichier de DAG neuf n'apparaît pas tout de suite. Le lecteur de DAG relit toutes les 30 secondes les fichiers qu'il connaît déjà, mais il ne cherche les nouveaux dans le dossier que toutes les 5 minutes : comptez jusqu'à 5 minutes avant que boreal_quotidien se montre. Dans Ubuntu :

```

docker compose exec airflow-scheduler airflow dags list
docker compose exec airflow-scheduler airflow dags list-import-errors

```

Le premier doit montrer boreal_quotidien, le second No data found. Si vous avez retouché le fichier entre deux lectures, boreal_quotidien peut revenir sur plusieurs lignes identiques : Airflow 3.3.1 affiche une ligne par version du DAG, et chaque changement de structure en crée une (un commentaire ajouté, non). Ce n'est pas un doublon. Lancez une exécution sur une journée qui a des données :

```

docker compose exec airflow-scheduler airflow dags trigger boreal_quotidien --logical-date 2026-03-15T00:00:00+00:00

```

Suivez-la dans l'interface, ou en ligne de commande. Vous y verrez aussi une exécution que vous n'avez pas lancée, celle de la veille : sans rattrapage, l'ordonnanceur ne crée que la dernière journée finie, et sa tâche raw ne trouve rien à charger, puisque les sources s'arrêtent au 31 mars 2026. Ce que les tâches de votre exécution écrivent, sur le poste de l'auteur :

Tâche	Ce qu'elle a fait
bronze	754 572 lignes réécrites en 35,7 s, fenêtre ignorée, l'étape le dit dans son journal
raw	fenêtre du 2026-03-15, 1 fichier, 949 lignes en 0,5 s

Tâche	Ce qu'elle a fait
corpus	1 document déposé en 0,9 s
dbt_build	rien, et c'est voulu

Les 35 secondes du bronze méritent une phrase. Le conteneur écrit sur des fichiers de Windows, à travers la frontière de WSL 2 : la même étape prend quelques secondes lancée directement sous Windows. C'est le coût de la traversée, connu depuis le Cours 05, et il ne change rien au résultat.

Piège — relancer la même journée. Airflow n'exécute une date qu'une fois par DAG : relancer la commande sur le 15 mars plante sur UniqueViolation, au bout d'une centaine de lignes de trace qui nomment dag_run_dag_id_logical_date_key. Pour rejouer, effacez d'abord ce que le DAG a gardé, par `airflow dags delete boreal_quotidien -y`, puis attendez qu'il revienne dans `airflow dags list`, une demi-minute environ : relancé trop tôt, `dags trigger` répond DagNotFound. L'effacement emporte l'historique des exécutions, pas les données de l'entrepôt.

Étape 9 — Préparer un témoin, écrire le DAG mensuel et laisser rattraper

Théorie : Theorie_06.md, sections 3 et 4.

Le second DAG ne fait qu'une chose, et c'est voulu : charger les extraits du mois de son intervalle. Dans l'Anaconda Prompt :

```
notepad 01_de\dags\boreal_mensuel.py
```

```
# -*- coding: utf-8 -*-
"""boreal_mensuel.py : Les livraisons mensuelles de Boréal.
```

Une tâche, qui charge dans la RAW les extraits du mois de son intervalle de données : Les neuf fichiers de l'ERP et les cinq des garanties.

La règle qu'il illustre : Le rattrapage. Mettez ce DAG en route avec trois mois de retard, et l'ordonnanceur crée trois exécutions, une par mois, chacune avec son intervalle. Le résultat doit être identique à un chargement des trois mois d'un seul bloc, et l'étape 10 le vérifie.

Pourquoi une seule tâche, alors que le quotidien en a quatre. Le mois n'apporte que des fichiers à charger : le bronze de l'usine est quotidien, le corpus n'a pas de rythme, et l'extraction de l'ERP par ODBC demande un pilote Windows, qui n'est pas dans l'image.

```
from __future__ import annotations
```

```
from datetime import timedelta
```

```
import pendulum
```

```
from airflow.sdk import DeadlineAlert, DeadlineReference, SyncCallback, dag
```

```

from airflow.providers.standard.operators.bash import BashOperator
from airflow.timetables.interval import CronDataIntervalTimetable

from boreal_alertes import prevenir_retard

PIPELINE = "cd /projet/01_de && python -m pipeline"
# Le mois de l'exécution, en AAAA-MM, Lu dans sa date logique : juste pour une exécution planifiée
# comme pour une exécution lancée à la main. La fenêtre accepte cette forme, et une borne de mois
# vaut tout le mois. Écrire les deux bornes laisse le DAG lisible quand on voudra un trimestre.
MOIS = "{{ logical_date.strftime('%Y-%m') }}"

@dag(
    dag_id="boreal_mensuel",
    # Une tranche par mois, en toutes lettres, comme dans le quotidien. L'exécution de janvier part
    # le 1er février, une fois le mois fini.
    schedule=CronDataIntervalTimetable("@monthly", timezone="UTC"),
    start_date=pendulum.datetime(2026, 1, 1, tz="UTC"),
    # La borne de fin est inclusive : mise au 2026-04-01, elle ferait naître une quatrième
    # exécution, celle d'avril. Elle ne planterait pas : le chargeur ne trouve aucun fichier dans
    # sa fenêtre, rend zéro ligne, et l'exécution passe au vert. Un mois vide déguisé en succès.
    end_date=pendulum.datetime(2026, 3, 31, tz="UTC"),
    catchup=True,
    max_active_runs=1,
    default_args={"retries": 1, "retry_delay": timedelta(minutes=5)},
    deadline=DeadlineAlert(
        reference=DeadlineReference.DAGRUN_QUEUED_AT,
        interval=timedelta(hours=2),
        callback=SyncCallback(
            prevenir_retard,
            kwargs={"message": f"boreal_mensuel du mois {MOIS} n'a pas fini en 2 heures"},
        ),
    ),
    tags=["boreal", "chemin froid"],
    doc_md=__doc__,
)
def boreal_mensuel():
    BashOperator(
        task_id="raw_du_mois",
        bash_command=f"{PIPELINE} raw --livraisons erp,garanties --depuis {MOIS} --jusqua {MOIS}",
        # Le même pool que le DAG quotidien : les deux écrivent le même fichier DuckDB.
        pool="entrepot",
    )

```

)

boreal_mensuel()

Avant de le laisser partir, fabriquez le témoin auquel vous comparerez. Il part du même état que votre entrepôt, et reçoit les trois mois d'un seul bloc. Dans l'Anaconda Prompt, depuis `Projet_Boreal` :

```
copy plateforme\entrepot\boreal.duckdb plateforme\entrepot\temoin.duckdb
cd 01_de
set PIPELINE_CHEMINS_ENTREPOT=plateforme/entrepot/temoin.duckdb
python -m pipeline raw --livraisons erp,garanties --depuis 2026-01 --jusqua 2026-03
set PIPELINE_CHEMINS_ENTREPOT=

fenêtre : du 2026-01-01 au 2026-03-31
livraisons : erp, garanties
RAW : 42 fichier(s) chargé(s) à ce passage
étape raw (d96d1a76) : fin, 20 664 lignes en 2.0 s
```

Quarante-deux fichiers, quatorze livraisons pour chacun des trois mois, 20 664 lignes, deux secondes. C'est la surcharge de configuration du Cours 05 qui vient de servir : `PIPELINE_CHEMINS_ENTREPOT` a détourné le pipeline vers un autre fichier, sans toucher au `config.toml`.

Piège — la variable qui reste. Sous Windows, `set` vaut pour toute la fenêtre de terminal. Si vous oubliez de la vider, tout le reste du labo écrira dans le témoin, et la comparaison de l'étape 10 comparera un fichier avec lui-même.

Maintenant, laissez l'ordonnanceur faire. Le compose crée les DAG non suspendus : celui-ci crée ses trois exécutions dès que le lecteur de DAG a trouvé le fichier, jusqu'à 5 minutes après son enregistrement, comme pour le quotidien. Dans Ubuntu :

```
docker compose exec airflow-scheduler airflow dags list-runs boreal_mensuel

run_id                                | state   | run_after                                | logical_date
scheduled__2026-04-01T00:00:00+00:00 | success | 2026-04-01T00:00:00+00:00              | 2026-03-01T00:00:00+00:00
scheduled__2026-03-01T00:00:00+00:00 | success | 2026-03-01T00:00:00+00:00              | 2026-02-01T00:00:00+00:00
scheduled__2026-02-01T00:00:00+00:00 | success | 2026-02-01T00:00:00+00:00              | 2026-01-01T00:00:00+00:00
```

Trois exécutions, une par mois ; la commande affiche aussi le DAG en tête de ligne, et les heures de début et de fin en queue. Lisez les deux dernières colonnes montrées. `run_after` est le moment où l'exécution a le droit de partir, une fois son mois fini, et c'est lui qui la nomme ; `logical_date` est le début du mois qu'elle traite. L'exécution de janvier s'appelle donc `scheduled__2026-02-01`, et c'est normal. Le rattrapage complet prend une trentaine de secondes, et chaque exécution charge exactement quatorze fichiers :

Mois	Fichiers	Lignes
2026-01	14	6 452

Mois	Fichiers	Lignes
2026-02	14	6 506
2026-03	14	7 706
Total	42	20 664

Les mêmes totaux que le témoin. C’est encourageant, mais ce n’est pas une preuve : deux comptes égaux ne disent rien du contenu.

Étape 10 — Prouver que le rattrapage égale le chargement d’un bloc

Théorie : Theorie_06.md, section 4.

Comparez les quatorze tables, ligne par ligne, en excluant la seule colonne qui doit différer : `_charge_le`, qui porte l’heure du chargement. Dans l’Anaconda Prompt, depuis `Projet_Boreal`, la ligne de commande DuckDB :

```
duckdb
```

À l’invite `D`, attachez les deux entrepôts, chacun sous son nom :

```
ATTACH IF NOT EXISTS 'plateforme/entrepot/boreal.duckdb' AS rattrape (READ_ONLY);
ATTACH IF NOT EXISTS 'plateforme/entrepot/temoin.duckdb' AS bloc (READ_ONLY);
```

Puis comparez une table, pour voir la forme de la question :

```
-- Attendu : 0 et 0, pour chacune des quatorze tables.
SELECT 'erp_vbak' AS table_raw,
       (SELECT COUNT(*) FROM (FROM (SELECT * EXCLUDE (_charge_le) FROM rattrape.raw.erp_vbak)
                                EXCEPT FROM (SELECT * EXCLUDE (_charge_le) FROM bloc.raw.erp_vbak))) AS rattrape_sans_bloc,
       (SELECT COUNT(*) FROM (FROM (SELECT * EXCLUDE (_charge_le) FROM bloc.raw.erp_vbak)
                                EXCEPT FROM (SELECT * EXCLUDE (_charge_le) FROM rattrape.raw.erp_vbak))) AS bloc_sans_rattrape;
```

Recommencer treize fois à la main serait pénible et fragile. Et le SQL ne sait pas boucler sur des noms de tables : un nom de table n’est pas une valeur, il ne se calcule pas dans une requête. Quand un contrôle doit passer sur une liste d’objets, il sort du SQL et rentre dans le code, qui peut lire cette liste dans `duckdb_tables()` et bâtir une requête par table. Créez-le :

```
notepad 01_de\comparer_rattrapage.py

# -*- coding: utf-8 -*-
"""comparer_rattrapage.py : La RAW rattrapée mois par mois contre la RAW chargée d'un bloc.
```

Les deux entrepôts partent du même état. L'un a reçu les trois mois de 2026 par trois exécutions du DAG mensuel, une par mois ; l'autre les a reçus d'un seul bloc. Les quatorze tables que le DAG

mensuel possède doivent être identiques, à la colonne _charge_Le près, qui porte l'heure du chargement et qui diffère par construction.

Lancement, depuis Projet_Boreal, l'entrepôt Libre : python 01_de/comparer_rattrapage.py

```
from pathlib import Path

import duckdb

ENTREPOT = Path("plateforme/entrepot/boreal.duckdb")
TEMOIN = Path("plateforme/entrepot/temoin.duckdb")
POSSEDEES = ("erp_", "garanties_")

con = duckdb.connect()
con.execute(f"ATTACH '{ENTREPOT.as_posix()}' AS rattrape (READ_ONLY)")
con.execute(f"ATTACH '{TEMOIN.as_posix()}' AS bloc (READ_ONLY)")

tables = [t[0] for t in con.execute(
    "SELECT table_name FROM duckdb_tables() "
    "WHERE database_name = 'rattrape' AND schema_name = 'raw' ORDER BY table_name").fetchall()]
possedees = [t for t in tables if t.startswith(POSSEDEES)]

print(f"{len(possedees)} tables du DAG mensuel, comparées sans la colonne _charge_le\n")
print(f"{'table':<28}{'rattrapé':>10}{'bloc':>10}{'écarts':>9}")
ecarts = 0
for table in possedees:
    a = f"(SELECT * EXCLUDE (_charge_le) FROM rattrape.raw.{table})"
    b = f"(SELECT * EXCLUDE (_charge_le) FROM bloc.raw.{table})"
    n_a = con.execute(f"SELECT COUNT(*) FROM {a}").fetchone()[0]
    n_b = con.execute(f"SELECT COUNT(*) FROM {b}").fetchone()[0]
    # EXCEPT ALL, et non EXCEPT : Le premier compte les exemplaires, Le second compare des
    # ensembles et écarte les doublons avant de comparer. Un mois chargé deux fois serait
    # invisible à EXCEPT, puisque ses lignes en trop sont identiques à celles du témoin.
    manque = con.execute(f"SELECT COUNT(*) FROM (FROM {a} EXCEPT ALL FROM {b})").fetchone()[0]
    surplus = con.execute(f"SELECT COUNT(*) FROM (FROM {b} EXCEPT ALL FROM {a})").fetchone()[0]
    # L'écart de comptes entre dans le verdict : une preuve qui ne peut pas échouer ne prouve rien.
    ecart_table = manque + surplus + abs(n_a - n_b)
    ecarts += ecart_table
    print(f"{'table':<28}{'n_a':>10}{'n_b':>10}{'ecart_table':>9}")

con.close()
print(f"\nVerdict : {'0 écart, le rattrapage égale le bloc' if ecarts == 0 else f'{ecarts} écarts'}")
```

Quittez DuckDB par `.exit`, puis lancez-le depuis `Projet_Boreal` :

```
python 01_de\comparer_rattrapage.py
```

14 tables du DAG mensuel, comparées sans la colonne `_charge_le`

table	rattrapé	bloc	écarts
erp_cdhdr	53	53	0
erp_cdpos	53	53	0
erp_kna1	7949	7949	0
erp_makt	5326	5326	0
erp_mara	2663	2663	0
erp_t005u	1040	1040	0
erp_vbak	28271	28271	0
erp_vbap	86524	86524	0
erp_zserie	34910	34910	0
garanties_clients	26826	26826	0
garanties_consentements	34683	34683	0
garanties_enregistrements	28118	28118	0
garanties_reclamations	7343	7343	0
garanties_visites	57583	57583	0

Verdict : 0 écart, le rattrapage égale le bloc

Quatorze tables, aucun écart dans un sens ni dans l'autre. Trois exécutions successives d'un ordonnanceur ont produit exactement ce qu'un chargement d'un seul bloc aurait produit. C'est la promesse du cours, et elle est maintenant mesurée plutôt que crue.

À retenir — ce que cette preuve autorise. Elle permet de rejouer un mois seul sans réfléchir, de remettre un DAG en route après une panne de plusieurs jours, et de reconstruire l'entrepôt de zéro en laissant le rattrapage travailler une nuit. Sans elle, chacune de ces trois opérations serait un pari.

Le témoin a fait son travail. Supprimez-le :

```
del plateforme\entrepot\tmoin.duckdb
```

Étape 11 — Versionner les DAG, la fenêtre et les tests

Dans l'Anaconda Prompt, depuis `Projet_Boreal` :

```
cd 01_de
git status
```

git status ne montre que ce qui est à vous : le dossier dags et ses deux DAG, la fenêtre ajoutée au paquet pipeline, et les deux tests de plus. Rien de la plateforme n'apparaît, et c'est voulu : son docker-compose.yml, son Dockerfile.airflow et les journaux qu'Airflow écrit vivent dans Projet_Boreal/plateforme, hors de votre dépôt. En entreprise, ils vivraient dans le dépôt de l'équipe qui exploite la plateforme, avec leur propre historique.

```
git add dags pipeline tests
git commit -m "Cours 06 : la fenetre du pipeline et les deux DAG"
git log --oneline
```

Récapitulatif

Ce que vous avez fait	Ce qui le prouve
donné au pipeline une tranche de temps	une journée de télémétrie chargée seule, 222 lignes ; trois garde-fous qui parlent
partagé les livraisons entre deux DAG	14 livraisons au mensuel, 8 au quotidien, aucune en double
vérifié le rattrapage sans Airflow	pytest rend 5 verts en une demi-seconde
rallumé Airflow 3 et lu sa pile	cinq services sains en une vingtaine de secondes, 1,1 Go
vérifié la protection de l'entrepôt entre DAG	un pool d'une place, et l'erreur de verrou qu'il évite
écrit deux DAG qui appellent le pipeline	le quotidien en quatre tâches, le mensuel en une
laissé l'ordonnanceur rattraper trois mois	42 fichiers, 20 664 lignes, trois exécutions vertes
prouvé que le rattrapage égale le bloc	14 tables, 0 écart dans les deux sens

Ce que l'entrepôt contient maintenant, et qui servira au Cours 07 :

Couche	Contenu	Compte
RAW	22 tables, l'historique jusqu'à mars 2026	580 907 lignes chargées en tout
bronze	l'historien et la télémétrie en Parquet, 182 jours d'usine	754 572 lignes
pipeline.chargements	une ligne par fichier chargé, avec son empreinte et son lot	

Couche	Contenu	Compte
pipeline.executions	une ligne par passage d'étape, DAG compris	

La table des exécutions mérite un coup d'œil : depuis ce cours, elle contient aussi ce qu'Airflow a lancé. Un pipeline qui se journalise lui-même reste lisible quel que soit celui qui appuie sur le bouton.

Dépannage

Symptôme	Cause probable	Correctif
No module named 'pipeline.fenetre'	le fichier s'appelle fenetre.py.txt	Notepad a ajouté l'extension : renommer, dir 01_de\pipeline montre le vrai nom
Livraison inconnue : garantie.	un « s » oublié, ou un préfixe inventé	la liste des connues est dans le message
« 2026-13 » n'est pas une date	une borne hors calendrier	les bornes sont AAAA-MM ou AAAA-MM-JJ
Fenêtre vide : --depuis ... vient après -jusqua ...	les deux bornes inversées	les remettre dans l'ordre
no configuration file provided: not found	docker compose lancé ailleurs que dans plateforme	dans Ubuntu, cd vers .../Projet_Boreal/plateforme
le DAG n'apparaît pas dans airflow dags list	un fichier neuf que le lecteur de DAG n'a pas encore trouvé, ou une erreur d'importation	attendre jusqu'à 5 minutes ; si rien ne vient, airflow dags list-import-errors nomme le fichier et la ligne
le même DAG sur plusieurs lignes de airflow dags list	une ligne par version du DAG : chaque changement de structure du fichier en crée une	rien à corriger
Provided callback ... is not awaitable	AsyncCallback reçoit une fonction ordinaire	SyncCallback pour une fonction, AsyncCallback pour une coroutine
Could not set lock on file ... Conflicting lock is held	deux tâches écrivent l'entrepôt en même temps	le pool entrepot d'une place, déclaré par chaque tâche qui appelle le pipeline
[Errno 1] Operation not permitted à l'étape corpus	shutil.copy2 recopie les métadonnées sur un montage Windows	shutil.copyfile dans pipeline/corpus.py

Symptôme	Cause probable	Correctif
une exécution met cinq minutes de plus que prévu	une tâche a raté une fois, et la nouvelle tentative a attendu son délai	lire le journal de la tâche : le premier essai dit pourquoi
airflow tasks clear s'arrête sur TypeError: unsupported type for timedelta seconds component: dict	une limite d'Airflow 3.3.1 sur les DAG qui portent une Deadline Alert	passer par airflow dags delete <dag_id> -y, puis relancer, ou effacer l'exécution depuis l'interface
airflow dags list-runs -d boreal_mensuel répond unrecognized arguments: -d	la ligne de commande a changé en Airflow 3	le DAG se donne en argument, sans -d
dags trigger plante sur UniqueViolation et dag_run_dag_id_logical_date_key	cette date a déjà une exécution dans ce DAG	airflow dags delete <dag_id> -y, attendre que le DAG revienne dans airflow dags list, puis relancer
DagNotFound: Dag id ... not found in DagModel juste après dags delete	le lecteur de DAG n'a pas encore relu le fichier	attendre une demi-minute
l'exécution planifiée de minuit charge la journée qui commence, vide	schedule="@daily" tout court : depuis Airflow 3, il ne découpe plus de tranche	schedule=CronDataIntervalTimetable("@daily", timezone="UTC")
un mois rejoué par dags trigger charge le mois du lancement	la commande lit data_interval_start, qu'Airflow calcule sur l'heure du lancement	lire la date logique : {{ ds }}, logical_date
l'interface répond 502 sur localhost:8080	le serveur d'API n'a pas fini de démarrer	docker compose ps : attendre que les cinq services soient healthy

Annexe — Tout le code, d'un seul tenant

Pour rejouer le labo sans relire les explications. Les commandes de terminal se tapent dans l'Anaconda Prompt, depuis Projet_Boreal, sauf celles des parties 4 et 6, dans Ubuntu.

1 — Les sources, régénérées (*Anaconda Prompt, depuis Data_Domain*)

```
python _univers/boreal/generer_boreal.py --projet Projet_Boreal --jusqua 2026-03
```

2 — Le pipeline (*créer 01_de\pipeline\fenetre.py, puis modifier raw.py, bronze.py, corpus.py, erp.py, meteo.py et __main__.py comme aux étapes 2 et 3*)

```
cd 01_de
python -m pipeline --etapes
python -m pipeline raw --livraisons telemetrie --depuis 2026-01-14 --jusqua 2026-01-14
python -m pipeline raw --livraisons telemetrie --depuis 2026-01-14 --jusqua 2026-01-14
pytest
cd ..
```

3 — Le témoin (*Anaconda Prompt*)

```
copy plateforme\entrepot\boreal.duckdb plateforme\entrepot\temoin.duckdb
cd 01_de
set PIPELINE_CHEMINS_ENTREPOT=plateforme/entrepot/temoin.duckdb
python -m pipeline raw --livraisons erp,garanties --depuis 2026-01 --jusqua 2026-03
set PIPELINE_CHEMINS_ENTREPOT=
```

4 — Airflow, déjà monté au bloc d'ouverture (*créer 01_de\dags ; puis, depuis Projet_Boreal, cd plateforme, wsl, et dans Ubuntu*)

```
docker compose start
docker compose ps
docker compose exec airflow-scheduler airflow pools list
```

5 — Les DAG (*créer 01_de\dags\boreal_alertes.py, boreal_quotidien.py et boreal_mensuel.py*)

6 — Les exécutions (*Ubuntu, depuis plateforme*)

```
docker compose exec airflow-scheduler airflow dags list
docker compose exec airflow-scheduler airflow dags list-import-errors
docker compose exec airflow-scheduler airflow dags trigger boreal_quotidien --logical-date 2026-03-15T00:00:00+00:00
docker compose exec airflow-scheduler airflow dags list-runs boreal_mensuel
```

7 — La preuve, et le ménage (*Anaconda Prompt, depuis Projet_Boreal*)

```
python 01_de\comparer_rattrapage.py
del plateforme\entrepot\temoin.duckdb
cd 01_de
git add dags pipeline tests
git commit -m "Cours 06 : la fenetre du pipeline et les deux DAG"
```